INSA INSTITUT NATIONAL DES SCIENCES APPLIQUÉES ROUEN NORMANDIE

Département ITI

Rapport de projet intégratif ITI 3.

Smart Robot

Étudiants

Maïssa	BEN SETHOUM	maissa.ben_sethoum@insa-rouen.fr
Lilian	DUJOURD'HUI	lilian.dujourdhui@insa-rouen.fr
Karim	HAMMOUDI	karim.hammoudi@insa-rouen.fr
Ilyass	EL ASSAD	ilyass.el_assad@insa-rouen.fr
Marwan	GUERROUACHE	marwan.guerrouache@insa-rouen.fr
Cristopher	ROYER	christopher.royer@insa-rouen.fr

1er octobre 2026

Synthèse du projet

Titre du projet : PI Smart Robot – Navigation d'un robot dans une ville suivant le plus court chemin

Résumé : Dans ce projet, nous avons conçu un robot autonome capable de parcourir une ville modélisée sous forme de cases numérotées, en suivant des lignes noires et en passant par des points de passage obligatoires. L'objectif est de permettre au robot de partir d'un point de départ, de passer par tous les points de passage et de revenir au point initial le plus rapidement possible. Le robot doit détecter les pastilles de couleur, émettre un bip à chaque passage et afficher les informations sur son écran.

Ce projet a mobilisé des compétences à la fois en électronique (cours d'électronique pour système embarqué) et en informatique (cours d'algorithme et programmation en C), autour de la conception et de la réalisation d'un robot autonome.

La partie électronique a consisté à concevoir la structure du robot, puis à la réaliser en tenant compte du matériel mis à disposition. Les différents composants ont été intégrés de manière à assurer le déplacement du robot et l'interaction avec son environnement. Des programmes de commande ont ensuite été développés afin de permettre au robot de se déplacer correctement, avant de procéder à une phase de tests et d'ajustements.

La partie informatique avait pour objectif principal de développer un algorithme permettant de déterminer le plus court chemin dans une ville modélisée. Pour mener à bien cette tâche, une démarche de cycle en V a été adoptée. Celle-ci a permis de structurer le travail autour d'une phase d'analyse, suivie d'une conception préliminaire, d'une conception détaillée, puis d'une phase de tests visant à valider le bon fonctionnement des algorithmes développés.

Ce projet a été réalisé en équipe, avec une répartition équitable des tâches en fonction des affinités et des compétences de chacun. Le travail s'est majoritairement effectué en binômes, dont la composition a pu évoluer au cours du projet, favorisant ainsi la collaboration et l'entraide entre les membres de l'équipe.

Mots clés : robot autonome, navigation, capteurs, Raspberry Pi, lignes noires, programmation, électronique.

Table des matières

Synthèse du projet **1**

1 Partie Informatique **4**
1.1 Mise en place du cycle en V 4
1.2 Analyse 4
1.2.1 Précision des TAD 4
1.2.2 Analyse descendante 7
1.3 Conception préliminaire 8
1.4 Conception détaillée et algorithmes non triviaux 9
1.5 Chargement de la ville 9
1.6 Recherche de chemin : `chercher_chemin` 11
1.6.1 Principe de l'algorithme 12
1.7 Traduction du chemin en ordres : `traduction_des_ordres` 13
1.8 Principe de l'algorithme 14
1.9 Implémentation en langage C 14
1.10 Conclusion Partie informatique 14

2 Partie Electronique **15**
2.1 Schéma électrique global 15
2.2 Explication détaillée des choix de câblage et techniques 15
2.3 Bilan de puissance et estimation de l'autonomie 15
2.4 Rôle et principe de fonctionnement des composants 16
2.5 Positionnement des composants et stratégie de détection 16
2.6 Difficultés rencontrées et pistes d'amélioration 17

3 Organigramme de démarche **17**

Introduction

Durant notre premier semestre en ITI 3, nous avons, en électricité, découvert les principes de l'électronique appliqués aux systèmes embarqués et appris comment les mettre en œuvre concrètement. Parallèlement, en cours d'Algorithme et Programmation en C, nous avons étudié les bases du langage C et approfondi notre culture algorithmique, en abordant notamment des algorithmes fondamentaux tels que celui de Dijkstra.

Le projet PI Smart Robot constitue l'opportunité de mettre en pratique ces connaissances théoriques et pratiques. Il nous permet de combiner électronique et programmation pour concevoir un système autonome capable de naviguer dans une ville modélisée, de détecter des points de passage et de réagir en temps réel. Ce projet favorise ainsi le développement de compétences transversales, alliant conception matérielle, programmation embarquée et résolution de problèmes complexes.

Au-delà des aspects techniques, ce projet nous a permis de développer des compétences importantes pour le travail en équipe. Réalisé à 6, il nous a appris à mieux collaborer, à nous répartir les tâches et à communiquer pour atteindre un objectif commun. Ce projet se rapproche également du fonctionnement du monde professionnel, notamment à travers la gestion de projet, l'organisation du travail et le respect des différentes étapes de réalisation. Enfin, le fait de mener ce projet de manière largement autonome, depuis l'analyse du problème jusqu'aux tests finaux, nous a permis de mieux comprendre l'ensemble des étapes nécessaires à la réalisation d'un projet d'ingénierie informatique, de A à Z.

1 Partie Informatique

Cette partie présente le raisonnement suivi pour aboutir au développement du programme principal, dont la sortie est une liste d'instructions que le robot utilisera en entrée pour se déplacer.

1.1 Mise en place du cycle en V

Afin de mener ce projet à bien, nous avons adopté une approche basée sur le cycle en V. Cette méthodologie a permis de structurer le développement logiciel en plusieurs étapes.

La première étape, l'analyse, a consisté à étudier le cahier des charges, à comprendre le fonctionnement attendu du robot et à identifier les besoins en termes de navigation et de détection des points de passage. À la suite de cette analyse, nous avons réalisé une analyse descendante permettant de schématiser la logique générale du programme ainsi que les types abstraits de données (TAD) qui seront utilisés.

La seconde étape, la conception préliminaire, a consisté à définir les signatures des fonctions et des procédures issues de l'analyse descendante, ainsi que celles associées aux différents TAD. La troisième étape, la conception détaillée, a permis de développer les algorithmes sous forme de pseudo-code.

Enfin, la phase de codage a consisté à implémenter en langage C l'ensemble des TAD, des fonctions et des procédures, ainsi que le programme principal.

1.2 Analyse

1.2.1 Précision des TAD

Lors de la phase d'analyse, nous avons d'abord cherché à identifier les informations indispensables pour résoudre le problème. Le fichier décrivant la ville fournit des données comme la largeur de la ville, la position et la direction initiales du robot, les connexions entre les cases ainsi que les points de passage obligatoires. Afin d'organiser correctement ces informations, nous avons choisi de définir des types abstraits de données adaptés.

Le TAD Case permet de représenter une case de la ville. Chaque case est identifiée par un numéro et contient des informations simples, comme le nombre de voisins accessibles. Ce choix est cohérent avec la modélisation de la ville sous forme de grille, où une case peut avoir jusqu'à quatre voisins.

TAD Case

Nom : Case
Utilise : Entier

Opérations

$$\text{creerCase} : \text{Entier} \to \text{Case} \tag{1}$$
$$\text{obtenirIdCase} : \text{Case} \to \text{Entier} \tag{2}$$
$$\text{obtenirNbVoisins} : \text{Case} \to \text{Entier} \tag{3}$$
$$\text{modIdCase} : \text{Case} \times \text{Entier} \to \text{Case} \tag{4}$$
$$\text{modNbVoisins} : \text{Case} \times \text{Entier} \to \text{Case} \tag{5}$$

Axiomes

Soit c une instance de `Case`, id un `Entier`, et nb un `Entier`.

1. $\text{obtenirIdCase}(\text{creerCase}(id)) = id$
2. $\text{obtenirNbVoisins}(\text{creerCase}(id)) = 0$
3. $\text{obtenirIdCase}(\text{modIdCase}(c, id)) = id$
4. $\text{obtenirNbVoisins}(\text{modIdCase}(c, id)) = \text{obtenirNbVoisins}(c)$
5. $\text{obtenirNbVoisins}(\text{modNbVoisins}(c, nb)) = nb$
6. $\text{obtenirIdCase}(\text{modNbVoisins}(c, nb)) = \text{obtenirIdCase}(c)$

Préconditions

— $\text{modNbVoisins}(c, nb)$: défini si et seulement si $0 \leq nb \leq 4$ (Selon la constante `MAX_VOISINS`)

Le TAD Ville regroupe toutes les informations globales nécessaires au programme, comme la largeur de la ville, le nombre total de cases, la position et la direction initiales du robot, ainsi que la liste des points de passage. Cette séparation entre les données liées aux cases et celles liées à la ville permet de mieux structurer le programme et de le rendre plus lisible.

TAD Ville

Nom : Ville
Utilise : Entier, Chaine, TableauEntiers, Case

Opérations

$$
\begin{aligned}
\text{creerVille} &: \text{Entier} \times \text{Entier} \to \text{Ville} \\
\text{obtenirDirectionInitiale} &: \text{Ville} \to \text{Chaine} \\
\text{obtenirPositionInitiale} &: \text{Ville} \to \text{Entier} \\
\text{obtenirLargeur} &: \text{Ville} \to \text{Entier} \\
\text{obtenirNbCases} &: \text{Ville} \to \text{Entier} \\
\text{obtenirPointsBip} &: \text{Ville} \to \text{TableauEntiers} \\
\text{obtenirNbPointsBip} &: \text{Ville} \to \text{Entier} \\
\text{modDirectionInitiale} &: \text{Ville} \times \text{Chaine} \to \text{Ville} \\
\text{modPositionInitiale} &: \text{Ville} \times \text{Entier} \to \text{Ville} \\
\text{modLargeur} &: \text{Ville} \times \text{Entier} \to \text{Ville} \\
\text{modNbCases} &: \text{Ville} \times \text{Entier} \to \text{Ville} \\
\text{modNbPointsBip} &: \text{Ville} \times \text{Entier} \to \text{Ville}
\end{aligned}
$$

Axiomes

Soit v une instance de `Ville`, d une `Chaine`, p, l, n, nb des `Entiers`.

1. $\text{obtenirDirectionInitiale}(\text{modDirectionInitiale}(v, d)) = d$
2. $\text{obtenirPositionInitiale}(\text{modPositionInitiale}(v, p)) = p$
3. $\text{obtenirLargeur}(\text{modLargeur}(v, l)) = l$
4. $\text{obtenirNbCases}(\text{modNbCases}(v, n)) = n$
5. $\text{obtenirNbPointsBip}(\text{modNbPointsBip}(v, nb)) = nb$
6. $\text{obtenirPositionInitiale}(\text{modDirectionInitiale}(v, d)) = \text{obtenirPositionInitiale}(v)$

Préconditions

— $\text{modNbCases}(v, n)$: défini si et seulement si $0 \leq n \leq$ `MAX_CASES` (où `MAX_CASES` = 100)
— $\text{modNbPointsBip}(v, nb)$: défini si et seulement si $0 \leq nb \leq$ `MAX_CASES`

1.2.2 Analyse descendante

Voici notre analyse descendante :

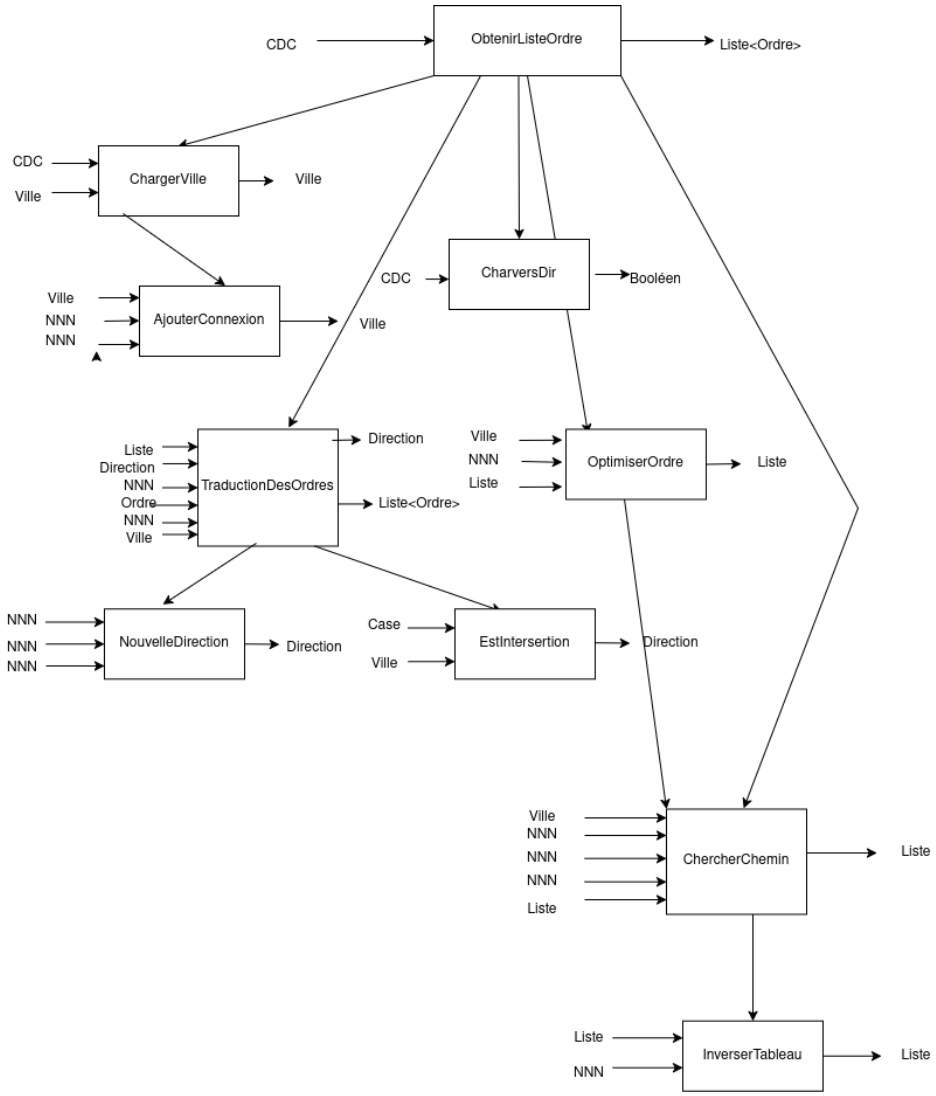


FIGURE 1 – Schéma Analyse Descendante

Après avoir défini les TAD, nous avons mis en place une analyse descendante afin d'organiser la logique générale du programme. L'objectif principal est de générer une liste d'ordres que le robot pourra exécuter à partir du fichier décrivant la ville. Pour cela, nous avons commencé par identifier une fonction principale chargée de produire cette liste d'instructions.

Cette fonction principale repose sur plusieurs sous-fonctions correspondant aux différentes étapes du traitement. Dans un premier temps, le programme doit lire le fichier et construire la représentation de la ville. Ensuite, il doit déterminer un chemin permettant de visiter tous les points de passage tout en minimisant la distance parcourue. Une fois ce chemin obtenu sous forme d'une suite de cases, il est nécessaire de le traduire en ordres simples compréhensibles par le robot, comme avancer ou tourner. Enfin, une étape d'optimisation permet de simplifier la liste d'ordres avant de la fournir au robot, celui-ci ne pouvant s'arrêter qu'au niveau des intersections.

1.3 Conception préliminaire

Lors de la conception préliminaire, nous avons repris les fonctions et procédures les plus importantes liées aux TAD et ajouté celles présentes dans l'analyse descendante.

Les signatures liées au TAD Case :

```
fonction obtenirNbVoisins( case : Case) : Entier;
procedure modNbVoisins( E/S case : Case,E nb Entier);
fonction obtenirIdCase(case : Case) Entier;
procedure modIdCase( E/S case : Case, E nb : Entier);
```

Les signatures liées au TAD Ville :

```
fonction obtenirDirectionInitialVille( ma_ville : Ville ) : Chaine De Caractère;
procedure modifierDirectionInitialVille( E/S ma_ville : Ville, E direction :
Chaine De Caractère);
fonction obtenirPositionInitialVille( ma_ville : Ville ) : Entier;
procedure modPositionInitialVille( E/S ma_ville : Ville, E position : Entier );
fonction largeurVille( ma_ville : Ville) : Entier;
procedure modLargeurVille( E/S ma_ville : Ville, E largeur : Entier );
fonction nbCaseVille( ma_ville : Ville ) : Entier;
procedure modNbCaseVille(E/S ma_ville : Ville, E nbCase : Entier );
fonction pointABip( ma_ville : Ville ) : tab [1... MAX_CASES] d'Entiers;
fonction NbPointABip( ma_ville : Ville ) : Entier;
procedure modNBPointABip( E/S ma_ville : Ville, E nbPoint : Entier);
```

Les signatures liées au chemin :

```
procedure inverserTableau( E/S tableau : Entier, E taille : Entier );
fonction chercherChemin( E/S ma_ville : Ville, E départ, arrivé, id_dejavu : Entier,
S chemin : Entier );
```

Nous avons également décidé d'ajouter des types de directions afin de répondre à la problématique consistant à générer une liste d'ordres. Les signatures associées sont donc :

```
fonction directionOppose( courante : Direction ) : Direction;
fonction nouvelleDirection(id_arrivee, id_depart, largeur_ville :Entier):Direction;
fonction charVersDir( direction : Chaine De Caractère );
```

Enfin nous avons besoins des algos de `Pilotage` :

```
fonction estIntersection(E case: Case, ville : Ville): Booléen;

procedure optimiserOrdre(E/S résultat : tab[1... MAX_CASES] d'Entiers, E ville:Ville,
départ : Entier, S min_étapes : Entier);
```

```
procedure traductionDesOrdes(E/S tableau_ordres : tab[1...MAX_CASES] de TypeOrdre,
nombre_ordres : entier E chemins : tab[1...MAX_CASES] d'entiers, direction_entrée :
Direction, longueur_chemin : Entier, ville : Ville, S directionActuelles: Direction);
```

1.4 Conception détaillée et algorithmes non triviaux

Le but de cette conception détaillée, est de produire les algorithmes en pseudo-code pour les types, fonctions et procédures les plus complexes, afin de pouvoir les traduire ultérieurement en langage C. Pour nos TAD, nous avons choisi d'utiliser des structures, qui nous semblent les plus adaptées. Elles permettent de stocker les informations de manière simple, logique et facilement accessible. Nous obtenons ainsi :

```
Constante MAX_VOISINS = 4;
Constante MAX_CASES = 100;

Type structure = Case :
    id : Entier ;
    voisins : tab [1... MAX_VOISINS] d'Entiers ;
    nb_voisins : Entier ;
Fin structure;

Type structure = Ville:
    largeur : Entier ;
    grille : tab [1... MAX_CASES] de Case ;
    nb_total_case : Entier ;
    position_initiale : Entier ;
    direction_initiale : Chaine De Caractères ;
    points_bip : tab [1... MAX_CASES] d'Entiers ;
    nb_points_bip : Entier ;
Fin structure;

Type Direction = [''NORD'', ''SUD'', ''EST'', ''OUEST''];

Type TypeOrdre = [''O_AV'', ''O_TD'', ''O_TG''];
```

1.5 Chargement de la ville

fonction `charger_ville`(E nom_fichier : Chaîne, E/S ma_ville : Ville) : Booléen
déclarations
f : Fichier
ligne : Chaîne
taille : Entier
position_initiale : Entier
direction_initiale : Chaîne
id1, id2 : Entier

```
    point_bip : Entier
début
    f ← ouvrir(nom_fichier, "r")
    si f = NULL alors
        afficherErreur("Erreur chargement ville")
        retourner Faux
    fin si

    // Initialisation de la structure (équivalent du memset)
    ma_ville ← creerVille(0,0)

    // Lecture taille
    si lireEntier(f, taille) alors
        si taille × taille > MAX_CASES alors
            fermer(f)
            retourner Faux
        fin si
        modLargeurVille(ma_ville, taille)
        modNbCaseVille(ma_ville, taille × taille)
    sinon
        fermer(f)
        retourner Faux
    fin si

    // Lecture position initiale
    si lireEntier(f, position_initiale) alors
        modPositionInitialVille(ma_ville, position_initiale)
    sinon
        fermer(f)
        retourner Faux
    fin si

    // Lecture direction initiale
    si lireChaine(f, direction_initiale) alors
        modDirectionInitialVille(ma_ville, direction_initiale)
    sinon
        fermer(f)
        retourner Faux
    fin si

    // Lecture du reste du fichier : connexions et points bip
    tant que lireLigne(f, ligne) faire
        si contient(ligne, "-") alors
            (id1, id2) ← extraireDeuxEntiersSeparesParTiret(ligne)
            ajouter_connexion(ma_ville, id1, id2)
        sinon
            si extraireEntier(ligne, point_bip) alors
                ajouterPointBipVille(ma_ville, point_bip)
            fin si
        fin si
    fin tant que
```

```
    fermer(f)
    retourner Vrai
fin
```

Le fichier passé en paramètre décrit la ville. Les deux premières lignes définissent la taille de la ville ainsi que la position et direction initiales du robot. Les lignes suivantes décrivent les connexions entre les cases à l'aide d'un tiret entre le numéros des cases. Enfin, les lignes qui ne contienent pas les tirets sont les points où il faut bipper.

1.6 Recherche de chemin : `chercher_chemin`

```
fonction chercher_chemin(
    E/S maville : Ville,
    E depart : Entier,
    E arrive : Entier,
    E id_interdit : Entier,
    S chemin : tab[1..Max_Cases] d'Entiers ) : Entier
déclarations
    file : tab[1..Max_Cases] d'Entiers
    queue, tete : Entier
    trouve : Booléen
    indice_depart, indice_arrive : Entier
    predecesseurs : tab[1..Max_Cases] d'Entiers
    deja_vu : tab[1..Max_Cases] d'Entiers
    actuel : Entier
    c : Case
    i : Entier
    id_voisin, indice_voisin : Entier
    longueur, courant : Entier
début
    queue ← 0
    tete ← 0
    trouve ← Faux
    // Initialisation des tableaux
    pour i de 1 à Max_Cases faire
        predecesseurs[i] ← -1
        deja_vu[i] ← 0
    fin pour
    indice_arrive ← arrive - 1
    indice_depart ← depart - 1
    // Enfiler le départ
    file[queue] ← indice_depart
    queue ← queue + 1
    tant que tete < queue faire // Pour parcourir toutes les cases
        actuel ← file[tete]
        tete ← tete + 1
        si actuel = indice_arrive alors
            trouve ← Vrai
```

```
                sortir
            fin si
            c ← maville.grille[actuel]
            pour i de 1 à c.nb_voisins faire
                id_voisin ← c.voisins[i]
                si id_voisin = id_interdit alors
                    continuer
                fin si
                indice_voisin ← id_voisin - 1
                si deja_vu[indice_voisin] = 0 alors
                    deja_vu[indice_voisin] ← 1
                    predecesseurs[indice_voisin] ← actuel
                    file[queue] ← indice_voisin
                    queue ← queue + 1
                fin si
            fin pour
        fin tant que

        si trouve = Faux alors
            retourner 0
        fin si

        // Reconstruction du chemin
        longueur ← 0
        courant ← indice_arrive
        tant que courant ≠ -1 faire
            chemin[longueur] ← maville.grille[courant].id
            longueur ← longueur + 1
            si courant = indice_depart alors
                sortir
            fin si
            courant ← predecesseurs[courant]
        fin tant que

        inverser_tableau(chemin, longueur)
        retourner longueur
fin
```

1.6.1 Principe de l'algorithme

La ville est modélisée par un graphe non pondéré dans lequel chaque case correspond à un sommet et les connexions sont représentées par une liste d'adjacence. Chaque sommet a donc la liste des cases voisines accessibles.

L'algorithme utilise un parcours en largeur (BFS) pour explorer le graphe sommet par sommet à partir de la case de départ. Une file est utilisée pour gérer l'ordre d'exploration alors que les tableaux `deja_vu` et `predecesseurs` permettent de ne pas revisiter une case et de mémoriser le chemin.

Le paramètre `id_interdit` est utilisé pour empêcher le robot de revenir immédiatement sur la case précédente afin d'éviter de faire un demi-tour. L'algorithme s'arrête dès que l'on atteint la case d'arrivée et permet de reconstruire le chemin à l'aide des

prédecesseurs, garantissant ainsi un chemin minimal en nombre de cases traversées.

1.7 Traduction du chemin en ordres : `traduction_des_ordres`

fonction `traduction_des_ordres`(
E chemins : tab d'Entiers,
E direction_entre : direction,
E longueur_chemin : Entier,
E ville : Ville,
S tab_ordres : tab de TypeOrdre,
E/S nb_ordres : Entier) : direction
déclarations
direction_actuelle, direction_souhaite : direction
i, j : Entier
case_actuelle, case_prochaine, case_test, case_suivante : Case
direction_du_saut, dir_suivante : direction
début
direction_actuelle ← direction_entre
i ← 0
tant que i < longueur_chemin - 1 **faire**
case_actuelle ← ville.grille[chemins[i] - 1]
case_prochaine ← ville.grille[chemins[i+1] - 1]
direction_souhaite ← nouvelle_direction(case_prochaine.id, case_actuelle.id, ville.largeur)
si direction_souhaite ≠ direction_actuelle **alors**
si (direction_actuelle = EST **et** direction_souhaite = SUD) **ou**
(direction_actuelle = NORD **et** direction_souhaite = EST) **ou**
(direction_actuelle = OUEST **et** direction_souhaite = NORD) **ou**
(direction_actuelle = SUD **et** direction_souhaite = OUEST) **alors**
tab_ordres[nb_ordres] ← O_TD
nb_ordres ← nb_ordres + 1
si direction_actuelle = NORD **alors** direction_actuelle ← EST **fin si**
si direction_actuelle = EST **alors** direction_actuelle ← SUD **fin si**
si direction_actuelle = SUD **alors** direction_actuelle ← OUEST **fin si**
si direction_actuelle = OUEST **alors** direction_actuelle ← NORD **fin si**
fin si
si (direction_actuelle = NORD **et** direction_souhaite = OUEST) **ou**
(direction_actuelle = OUEST **et** direction_souhaite = SUD) **ou**
(direction_actuelle = SUD **et** direction_souhaite = EST) **ou**
(direction_actuelle = EST **et** direction_souhaite = NORD) **alors**
tab_ordres[nb_ordres] ← O_TG
nb_ordres ← nb_ordres + 1
si direction_actuelle = NORD **alors** direction_actuelle ← OUEST **fin si**
si direction_actuelle = OUEST **alors** direction_actuelle ← SUD **fin si**
si direction_actuelle = SUD **alors** direction_actuelle ← EST **fin si**
si direction_actuelle = EST **alors** direction_actuelle ← NORD **fin si**
fin si
fin si

```
            tab_ordres[nb_ordres] ← O_AV
            nb_ordres ← nb_ordres + 1

            j ← i + 1
            direction_du_saut ← direction_souhaite
            tant que j < longueur_chemin - 1 faire
                case_test ← ville.grille[ chemins[j] - 1 ]
                si est_intersection(case_test, ville) alors
                    sortir
                fin si
                case_suivante ← ville.grille[ chemins[j+1] - 1 ]
                dir_suivante ← nouvelle_direction(case_suivante.id, case_test.id, ville.largeur)
                si dir_suivante ≠ direction_du_saut alors
                    sortir
                fin si
                j ← j + 1
            fin tant que
            i ← j
        fin tant que

        retourner direction_actuelle
fin
```

1.8 Principe de l'algorithme

Cette fonction transforme un chemin (suite de cases) en une suite d'ordres moteurs `O_TG`, `O_TD` et `O_AV`. À chaque étape, la direction à suivre est calculée avec `nouvelle_direction`. Si la direction souhaitée diffère de la direction actuelle, on ajoute d'abord un virage (gauche ou droite), puis un ordre d'avance.

Après un `O_AV`, l'algorithme regroupe plusieurs ordres `O_AV` tant que le robot reste dans la même direction et qu'il n'arrive pas à une intersection (test `est_intersection`).

1.9 Implémentation en langage C

Les algorithmes décrits dans la conception détaillée ont été implémentés en langage C. Le chargement de la ville est implémenté dans le fichier `Ville.c`, la recherche de chemin par BFS dans `Chemin.c`, et la traduction du chemin en ordres moteurs dans `pilote.c`.

1.10 Conclusion Partie informatique

Dans la phase de conception, l'accès aux données est présenté via des opérations d'encapsulation afin de mettre en évidence les abstractions. L'implémentation en langage C accède directement aux structures pour des raisons de simplicité et d'efficacité, tout en conservant la même logique algorithmique.

2 Partie Electronique

Cette section présente la conception matérielle du robot, les choix de câblage effectués pour garantir la fiabilité du système, ainsi que l'analyse du fonctionnement des composants sélectionnés.

2.1 Schéma électrique global

Le schéma électrique complet du robot est présenté dans un autre fichier. Il détaille les interconnexions entre la Raspberry Pi, le contrôleur moteur L293D, ainsi que l'ensemble des capteurs et actionneurs.

2.2 Explication détaillée des choix de câblage et techniques

Pour répondre aux contraintes d'intégration, nous avons structuré la distribution de l'énergie autour de connecteurs de type **Wago** (faisant office de dominos). Ces derniers sont utilisés comme des rails d'alimentation centralisés pour distribuer le 3.3V et la masse (GND) à l'ensemble des modules. Ce choix permet de réduire l'encombrement du câblage et de pallier le nombre limité de broches d'alimentation de la Raspberry Pi.

Une attention particulière a été portée à la gestion des masses. Bien que le robot dispose de deux sources d'énergie distinctes une batterie USB-C pour la Raspberry Pi et deux batteries de 4,5V pour les moteurs un point de **masse commune (GND)** a été établi sur l'un des Wago. Cette référence commune est indispensable pour que les signaux de commande PWM émis par la Raspberry Pi soient correctement interprétés par le contrôleur moteur L293D.

Concernant l'adaptation des niveaux logiques, l'ensemble des capteurs de ligne TCRT5000 a été alimenté en 3.3V. Ce choix technique permet de relier directement les sorties numériques des capteurs aux broches GPIO de la Raspberry Pis. Pour la protection du circuit, nous nous appuyons exclusivement sur les diodes et résistances déjà intégrées.

Enfin, conformément au cahier des charges, un interrupteur a été inséré dans le circuit de puissance. Il permet de couper l'alimentation des moteurs tout en maintenant la Raspberry Pi allumée, facilitant ainsi les phases de test logiciel sans risque de mouvement non souhaité.

2.3 Bilan de puissance et estimation de l'autonomie

Le bilan énergétique repose sur la consommation nominale des composants actifs listés dans le cahier des charges. La consommation maximale est atteinte lorsque les moteurs fonctionnent à plein régime, que l'écran LCD est allumé et que le buzzer est utilisé.

Paramètre	**Valeur**
Capacité totale des batteries	842 mAh
Consommation moyenne estimée	240 mA
Autonomie théorique estimée	**4.21 heures**

Table 1 – Synthèse du bilan de puissance et de l'autonomie.

2.4 Rôle et principe de fonctionnement des composants

Le contrôleur L293D : Ce composant est un double pont en H permettant de piloter le sens de rotation et la vitesse des deux moteurs. La vitesse est modulée par un signal PWM (Pulse Width Modulation) généré par le programme `parcours.c` via la fonction `pwmWrite`.

Capteurs de ligne (TCRT5000) : Ces capteurs infrarouges fonctionnent par émission et réception d'un signal lumineux. La surface noire de la piste absorbant le signal, le capteur renvoie un état logique haut (1) ou bas (0) selon la réflexion détectée. Ces capteurs ont donc dû être réglé en conséquence.

Capteur de couleur (TCS34725) : Connecté via le bus I2C, ce capteur permet d'identifier les pastilles vertes et rouge. Il convertit la lumière reçue en données numériques R, G, B et Clear qui sont ensuite traitées par le thread dédié `thread_detection_couleur`.

Buzzer et Écran LCD : Le buzzer émet un signal sonore lors de la détection d'une pastille verte. L'écran LCD affiche en temps réel l'instruction de navigation et la couleur détectée.

2.5 Positionnement des composants et stratégie de détection

L'emplacement des composants a été pensé pour optimiser la réactivité du robot. Les quatre capteurs de ligne sont positionnés à l'avant : deux centraux assurent le suivi de la ligne, tandis que deux latéraux permettent d'anticiper les intersections. Le capteur de couleur est quand a lui fixé sous le robot, vers l'avant, pour identifier les pastilles.

Pour garantir une réactivité optimale, la gestion des pastilles vertes s'effectue via un système de **multithreading** en parallèle et en continu. En revanche, la détection de la pastille rouge est réservée à la dernière instruction du parcours (`suivit_ligne_avancer_detection_rou`) pour signaler l'arrêt définitif. Le buzzer est placé à l'intérieur du robot, son emplacement précis n'impactant pas le fonctionnement.

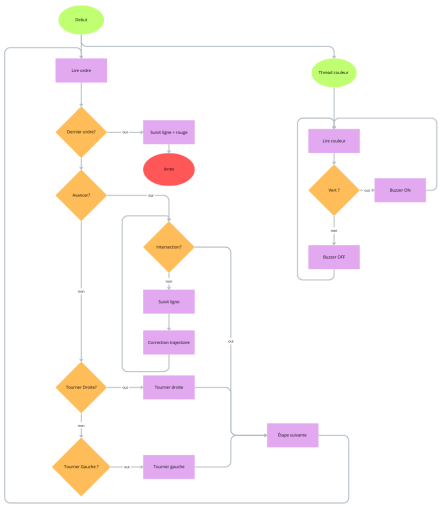


FIGURE 2 – Organigramme fonctionnel du robot PI Smart.

2.6 Difficultés rencontrées et pistes d'amélioration

La principale difficulté a résidé dans la gestion des alimentations multiples. Une autre problématique majeure concernait la gestion séquentielle des ordres d'avancement : il a fallu s'assurer que si le robot doit avancer deux fois de suite, il ne s'arrête pas sur la première intersection détectée par erreur de logique.

De plus, la gestion des virages a posé des difficultés, car leur précision dépend fortement de la vitesse de rotation des roues et de la position exacte du robot lors de l'arrêt à l'intersection.

Comme pistes d'amélioration, nous pourrions envisager :

— Un code plus portable en remplaçant les instructions `delay` par une gestion temporelle non bloquante.
— Une optimisation du placement des capteurs pour une meilleure anticipation des courbes.
— Une amélioration de l'interface LCD pour afficher davantage d'informations télémétriques.
— Un algorithme de reconnaissance de couleurs plus robuste, capable d'enregistrer une plus large gamme de teintes sans nécessiter de calibrage manuel systématique.

3 Organigramme de démarche

Tâches / Membres	**Lilian**	**Maïssa**	**Marwan**	**Ilyass**	**Karim**	**Christopher**
Analyse des besoins	X	X	X	X	X	X
Conception préliminaire				X		X
Conception détaillée (algorithmes)			X	X		
Développement C (logique robot)	X	X	X	X		
Développement C (capteurs / moteurs)	X	X	X	X	X	X
Tests unitaires			X	X	X	
Documentation Doxygen				X		
Rédaction du rapport	X	X	X	X	X	X
Montage du robot	X					X
Schéma électrique						X
Bilan de puissance+autonomie						X

TABLE 2 – Matrice de répartition des tâches au sein de l'équipe

Le tableau suivant nous montre comment nous nous sommes organisés durant les tâches.

Conclusion

La réalisation de ce projet a permis de concevoir et de valider un robot autonome capable de suivre un trajet imposé et de passer par des points spécifiques du labyrinthe, tout en exécutant une action de type bip lors de leur franchissement. Malgré les contraintes techniques et temporelles, le robot a démontré un fonctionnement globalement fiable, notamment en ce qui concerne le suivi du parcours, la détection des points requis et la coordination entre les différentes composantes matérielles et logicielles.

L'un des objectifs majeurs du projet consistait à optimiser le déplacement du robot en lui faisant emprunter le plus court chemin possible à travers le labyrinthe. Bien que cette approche ait été étudiée et amorcée sur le plan algorithmique, elle n'a pas pu être pleinement exploitée dans notre implémentation finale. Des difficultés liées à la complexité de la gestion du graphe, à l'intégration avec le contrôle du robot et aux contraintes de temps ont limité la faisabilité de cette optimisation. Face à ces obstacles, nous avons fait le choix pragmatique de privilégier une solution fonctionnelle et robuste, garantissant le passage par l'ensemble des points imposés, même si le chemin parcouru n'était pas optimal en termes de distance.

Ce projet a ainsi mis en évidence l'écart qui peut exister entre une solution théoriquement optimale et une solution réellement opérationnelle dans un système embarqué. Il nous a permis de mieux comprendre les enjeux liés à l'intégration d'algorithmes de recherche de chemin dans un contexte réel, où les contraintes matérielles, la précision des capteurs et la fiabilité de l'exécution jouent un rôle déterminant.

En conclusion, ce travail a constitué une expérience formatrice tant sur le plan technique que méthodologique. Il a renforcé notre compréhension des systèmes robotiques, de la programmation embarquée et de la gestion de projet en équipe.

Les pistes d'amélioration futures pourraient inclure une meilleure optimisation du calcul du plus court chemin, une gestion plus avancée des erreurs et une intégration plus fine entre la planification du trajet et le contrôle du robot, afin d'atteindre un comportement à la fois optimal et robuste.